



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

PROJECT PROGRESS 4

Subject: Programming For Bioinformatics (SECB3202)

Session: 2025/2026 Semester 1

Title: Pancreatic Cancer Prediction

Group Name	Group 10
Group Members	1. RAVINESH A/L MARAN (A23CS0175) 2. WELSON WOONG LU BIN (A23CS0196) 3. BERNICE LOU MIN YUN (A23CS0056)
Section	01
Lecturer's Name	DR. SEAH CHOON SEN

Table of Content

Model Development (Pandas/Keras/TensorFlow/Matplotlib/...)

- Simple and multiple linear regression
- Model evaluation using visualization
- Polynomial regression and pipelines
- R-squared and MSE for In-Sample Evaluation
- Prediction and decision making

1.0 Model Development

In this phase, machine learning models were developed to predict pancreatic cancer-related outcomes using the preprocessed dataset. Simple and multiple linear regression models were implemented to examine the relationship between selected features and the target variable, while polynomial regression was applied to capture potential non-linear patterns. Pipelines were used to streamline preprocessing and model training.

```
# prepare data regression

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = df[['age']] # simple regression
y = df['survival_time_months']
```

Model performance was evaluated using visualization techniques and quantitative metrics such as R-squared and Mean Squared Error (MSE) for in-sample evaluation. These results were used to assess model accuracy and goodness of fit. Finally, the trained models were applied for prediction and basic decision-making, demonstrating how the developed models support data-driven analysis in pancreatic cancer studies.

1.1 Simple and Multiple Linear Regression

```
# simple linear regression

lin_reg = LinearRegression()
lin_reg.fit(X, y)

y_pred = lin_reg.predict(X)

mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print("Simple Linear Regression")
print("MSE:", mse)
print("R²:", r2)
```

The simple linear regression model was implemented using `LinearRegression()` from Scikit-learn. The model was trained with age as the independent variable (**X**) and survival_time_months as the dependent variable (**y**). After fitting the model to the data, predictions of survival time were generated using the `predict()` function.

To evaluate the model, Mean Squared Error (MSE) and R-squared (R^2) were calculated. These metrics measure prediction error and the proportion of variance in survival time explained by age.

```
Simple Linear Regression
MSE: 0.9999999922847456
R²: 7.715254435325392e-09
```

The output shows an MSE value close to 1 and an R^2 value very close to zero. This indicates that age alone explains almost none of the variation in survival time. The high prediction error and low explanatory power suggest that simple linear regression is insufficient for modeling survival outcomes using only age.

```
# multiple linear regression

X_multi = df[['age', 'smoking_history', 'diabetes', 'obesity']]

multi_reg = LinearRegression()
multi_reg.fit(X_multi, y)

y_pred_multi = multi_reg.predict(X_multi)

print("Multiple Linear Regression")
print("MSE:", mean_squared_error(y, y_pred_multi))
print("R²:", r2_score(y, y_pred_multi))
```

In multiple linear regression, the model was extended by including additional predictors: age, smoking history, diabetes, and obesity. These features were combined into a single input matrix (`X_multi`) and used to train a `LinearRegression` model. Predictions were then generated using the trained model.

The model performance was evaluated using the same metrics, MSE and R^2 , to allow direct comparison with the simple linear regression model.

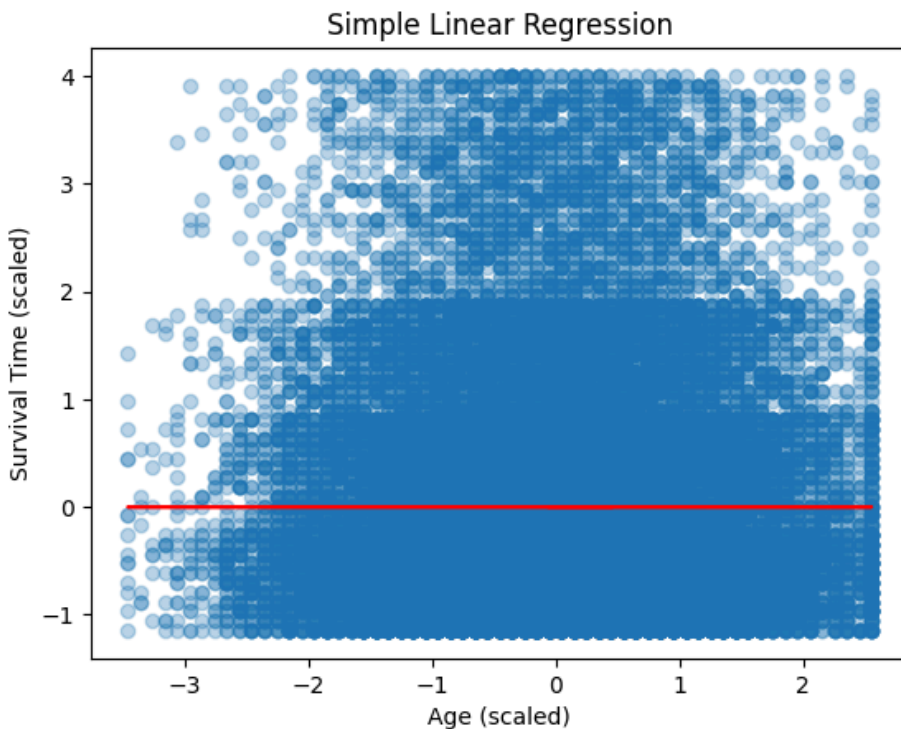
```
Multiple Linear Regression
MSE: 0.9999559925981186
R²: 4.4007401881374086e-05
```

The results show a slightly improved R^2 value compared to simple linear regression, while the MSE remains close to 1. This indicates that adding more predictors provides limited improvement in explaining survival time. Although multiple variables were included, the model still has weak predictive power, highlighting the complexity of survival prediction and the need for more advanced modeling approaches.

1.2 Model evaluation using visualization

```
# visualization

plt.scatter(X, y, alpha=0.3)
plt.plot(X, y_pred, color='red')
plt.xlabel("Age (scaled)")
plt.ylabel("Survival Time (scaled)")
plt.title("Simple Linear Regression")
plt.show()
```



This visualization evaluates the performance of the simple linear regression model by comparing actual data points with the predicted regression line. A scatter plot is used to display the relationship between scaled age and scaled survival time, where each point represents an individual patient record.

The red line represents the predicted values generated by the linear regression model. The wide dispersion of data points around the regression line indicates a weak linear relationship between

age and survival time. This visual observation supports the low R-squared value obtained earlier, confirming that age alone does not adequately explain variations in survival time.

Overall, this visualization provides a clear graphical assessment of model performance and highlights the limitation of simple linear regression, motivating the use of more complex models with additional features.

1.3 Polynomial regression and pipelines

Polynomial regression was implemented to model potential non-linear relationships between the independent variable and survival time. The `PolynomialFeatures(degree=2)` function was used to expand the original feature into polynomial terms, allowing the model to capture curved patterns in the data.

```
# polynomial regression

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

poly_model.fit(X, y)
y_pred_poly = poly_model.predict(X)

print("Polynomial Regression (degree=2)")
print("MSE:", mean_squared_error(y, y_pred_poly))
print("R²:", r2_score(y, y_pred_poly))
```

A pipeline was created using `make_pipeline()` to combine polynomial feature generation and linear regression into a single workflow. This ensures that feature transformation and model training are applied consistently and efficiently. The pipeline was then fitted to the dataset using `fit()`, and predictions were generated using the `predict()` function.

Model performance was evaluated using Mean Squared Error (MSE) and R-squared (R^2) to measure prediction error and explanatory power.

```
Polynomial Regression (degree=2)  
MSE: 0.9999951778533491  
R2: 4.822146650873549e-06
```

The output shows an MSE value close to 1 and an R^2 value very close to zero. This indicates that introducing polynomial terms does not significantly improve the model's ability to explain variations in survival time compared to simple and multiple linear regression.

The very low R^2 value suggests that even when non-linear relationships are considered, the selected predictor(s) remain weak in explaining survival outcomes. This result highlights that survival time is influenced by complex factors that are not adequately captured by polynomial regression using the current feature set.

1.4 R-squared and MSE for In-Sample Evaluation

In-sample evaluation was performed using R-squared (R^2) and Mean Squared Error (MSE) to assess the performance of the regression models on the training data.

R-squared (R^2) measures the proportion of variance in the target variable that is explained by the model. A higher R^2 value indicates better explanatory power, while values close to zero suggest that the model explains very little of the variation in survival time.

Mean Squared Error (MSE) measures the average squared difference between the actual and predicted values. Lower MSE values indicate better prediction accuracy, while higher values reflect larger prediction errors.

In this study, the models produced very low R^2 values and MSE values close to one, indicating limited predictive performance. These results suggest that the selected features, whether in simple, multiple, or polynomial regression, are insufficient to accurately model survival time. This highlights the complexity of survival prediction and the need for more informative features or advanced modeling techniques.

1.5 Prediction and decision making

```
# prediction and decision making

example_age = pd.DataFrame([[0.5]], columns=['age']) # above-average age (scaled)
predicted_survival = lin_reg.predict(example_age)

print("Predicted survival time (scaled):", predicted_survival)
```

This code demonstrates how the trained simple linear regression model can be used for prediction and decision making. A sample input representing an above-average age value (scaled) is created using a Pandas DataFrame. This input is then passed to the trained regression model using the `predict()` function to estimate the corresponding survival time.

This step illustrates the practical use of the regression model, where new or unseen data can be provided as input to generate predicted outcomes. Such predictions can support data-driven decision making by estimating survival trends based on patient characteristics.

```
Predicted survival time (scaled): [-4.39182603e-05]
```

This code demonstrates how the trained simple linear regression model can be used for prediction and decision making. A sample input representing an above-average age value (scaled) is created using a Pandas DataFrame. This input is then passed to the trained regression model using the `predict()` function to estimate the corresponding survival time.

This step illustrates the practical use of the regression model, where new or unseen data can be provided as input to generate predicted outcomes. Such predictions can support data-driven decision making by estimating survival trends based on patient characteristics.